Combining Services, Networks, and Volumes in Docker Compose

In this train, we'll combine services, networks, and volumes into a cohesive docker-compose.yml file ready for deployment. Through practical exercises, we'll integrate all these components to orchestrate a complete application stack.

Learning objective

> - Be able to combine services, networks, and volumes into a cohesive docker-compose.yml file ready for deployment.

Prerequisites

- Docker installed on your machine
- Basic understanding of Docker concepts (containers, images, networks, volumes)
- Familiarity with YAML syntax

Understanding the Core Components

Before diving into the practical exercises, let's recap the key components:

- **Services:** Define the containers that run your applications (e.g., web server, database).
- **Networks:** Enable communication between services (default: bridge network).
- **Volumes:** Persist data beyond the container lifecycle (e.g., database storage).

Practical Exercise: Multi-Service Application

We'll build a simple web application + database stack using:

- Nginx (web server)
- Node.js (backend API)
- PostgreSQL (database)
- Custom network for secure communication
- Named volumes for database persistence

Step 1: Create the `docker-compose.yml` File

```
version: '3.8'

services:
  # Web Server (Nginx)
  web:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    networks:
      - app-network
    depends_on:
      - api

  # Backend API (Node.js)
  api:
    build: ./api  # Assumes a Dockerfile in ./api
    environment:
      - DB_HOST=db
      - DB_USER=postgres
      - DB_PASSWORD=secret
    networks:
      - app-network
    depends_on:
      - db

  # Database (PostgreSQL)
  db:
    image: postgres:13
    environment:
      - POSTGRES_PASSWORD=secret
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-network

# Define a custom network
networks:
  app-network:
    driver: bridge

# Define a named volume for database persistence
volumes:
  db-data:
```

Step 2: Explanation of Key Sections

Services

- `web`: Uses Nginx to serve static files or proxy requests to the API.
- `api`: A Node.js app (requires a Dockerfile in ./api).
- `db`: PostgreSQL with persistent storage via a named volume.

Networks

- A custom `app-network` ensures secure communication between services.

Volumes

- `db-data` ensures PostgreSQL data persists even if the container restarts.

Dependencies

- `depends_on` ensures services start in the correct order.

Step 3: Create Supporting Files

1. `./api/Dockerfile (Example)`

```
FROM node:14
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

2. `./nginx.conf` (Basic Reverse Proxy)

```
events {}
http {
  server {
    listen 80;
    location / {
      proxy_pass http://api:3000;
    }
  }
}
```

Step 4: Deploy the Stack

Run: `docker-compose up -d`
Verify services: `docker-compose ps`

Step 5: Clean Up

Run: `docker-compose down -v`

Conclusion

Congratulations! You've successfully combined services, networks, and volumes in Docker Compose.

Key Takeaways

> - Services can be linked via `networks` for secure communication.
> - `volumes` ensure data persists beyond container restarts.
> - `depends_on` controls startup order (but not service readiness).
> - Custom configurations (like Nginx) can be mounted via bind mounts.